

עצירת מזיגת מים מבוסס הקלטה אקוסטית

קורס: מבוא ללמידת מכונה

מרצה: ד"ר דימה בוחובסקי

מגישים: אור בסטקר, 325834232

תמיר גרוסמן, 325668580

עמית ניגרקר, 207244658

יובל חולואידובסקי, 322686171

<https://github.com/TamirGrossman/Cup-Water-Level-Detection>

תוכן עניינים

1.....	עצירת מזיגת מים מבוסס הקלטה אקוסטית
2.....	תוכן עניינים
3.....	תקציר
4.....	מבוא
5.....	אופן ביצוע ההקלטות ומדידת המים
5.....	קטע קוד:
8.....	רקע תיאורטי
8.....	Feature Selection
9.....	שיטות לבחירת מאפיינים:
16.....	מודל XGBoost (eXtreme Gradient Boosting)
17.....	Cross-Validation
20.....	MFCC — Mel-Frequency Cepstral Coefficients
21.....	Hyperparameter Tuning
23.....	Random Forest
25.....	Stratified Group K-Fold
26.....	שילוב המודלים XGBoost ו-Random Forest
28.....	מבנה הקוד:
29.....	מדדי הערכה
30.....	תוצאות הקוד:
30.....	איור 1: דירוג המאפיינים האקוסטיים לפי ANOVA F-score
31.....	איור 2: השפעת מספר המאפיינים על דיוק מודל Random Forest
32.....	איור 3: השפעת מספר המאפיינים על דיוק מודל XGBoost
33.....	איור 4: מטריצת בלבול עבור מודל Random Forest בתהליך K-Fold
34.....	איור 5: מטריצת בלבול עבור מודל XGBoost בתהליך K-Fold
35.....	איור 6: מטריצת בלבול עבור שילוב המודלים XGBoost ו-Random Forest
36.....	איור 7: דוח מדדי ביצוע עבור המודל המשולב
37.....	איור 8: השוואת ביצועי XGBoost ו-Random Forest באמצעות Stratified 4-Fold Cross-Validation
38.....	איור 9: השוואה בין קצב זרימת מים אמיתי לבין קצב זרימה חזוי
39.....	איור 10: ספקטרוגרמה וחילוץ תדרים דומיננטיים מתוך הקלטת האודיו
40.....	סיכום ומסקנות כלליות
41.....	מקורות וספרות עזר

תקציר

פרויקט זה מציג פיתוח והוכחת קונספט למערכת אוטומטית המזהה את אחוז מילוי המים בכוס בעת הזרמתם, על בסיס ניתוח אותות אקוסטיים. הפתרון נשען על התופעה הפיזיקלית לפיה צליל המילוי משתנה באופן רציף עם השתנות הנפח והעומק. מטרת האלגוריתם היא לנתח את ספקטרום התדרים ולסווג את טווח המילוי הנוכחי מתוך חמישה טווחים מוגדרים: עד 20%, 20%-40%, 40%-60%, 60%-80%, ו-80%-100%.

לצורך כך, הוקם מסד נתונים הכולל עשרות הקלטות מגוונות מכוסות בגדלים ומחומרים שונים, תחת תנאי סביבה מבוקרים, מרחק קבוע ומצב התחלתי של כוס ריקה. האותות עברו עיבוד מקדים והתמרה למרחב התדר. בשלב הלמידה, בוצע תהליך של בחירת מאפיינים (Feature Selection) לזיקוק רכיבי התדר המשמעותיים ביותר ומניעת מורכבות חישובית. מאפיינים אלו שימשו כקלט למודל XGBoost ולמודל Random Forest ובוצעה הצלבה ושילוב ביניהם. תיקופו בוצע באמצעות מתודולוגיית תיקוף צולב (k-fold Cross Validation).

הערכת הביצועים הראתה כי המערכת הגיעה ליכולת הכללה גבוהה ודיוק מירבי בסיווג הטווחים, ללא תלות בסוג הכוס. תוצאות אלו מוכיחות כי שילוב של ניתוח ספקטרלי עם למידת מכונה קלאסית מספק פתרון יציב ומבוסס מספרים, המהווה תשתית הנדסית איתנה להמשך פיתוח המערכת.

מבוא

לניתוח ועיבוד אותות אקוסטיים ישנם יישומים נרחבים ומגוונים בטכנולוגיה המודרנית, החל מזיהוי דיבור ומידול חיזוי, ועד לאוטומציה תעשייתית ומכשירים ביתיים חכמים. בפרויקט זה, המבוצע כחלק מקורס למידת מכונה אנו חוקרים פיתוח ויישום של מערכת אוטומטית שנועדה לסווג את רמת המים בתוך כוס במהלך תהליך המילוי.

העיקרון המדעי שבבסיס עבודה זו נשען על התופעה הפיזיקלית לפיה התדרים האקוסטיים והתהודה הנוצרים בעת מילוי מיכל משתנים באופן רציף ככל שנפח הנוזל עולה ועמוד האוויר העליון קטן. באמצעות כלים ממוחשבים ושיטות לעיבוד אותות, האלגוריתם קולט את השינויים האקוסטיים הרציפים הללו, מתמיר אותם למרחב התדר ומחיל תהליך של בחירת מאפיינים (Feature Selection) כדי לזקק את רכיבי הספקטרום המובהקים ביותר.

במקום להסתמך על רשתות נוירונים עמוקות, הפרויקט נצמד באופן עקבי למתודולוגיות קלאסיות של למידת מכונה. באופן ספציפי, מודל XGBoost ומודל Random Forest לצד הצלבה ושילוב ביניהם, למשימת סיווג מרובה מחלקות (Multi-class) מאומנים ומוערכים באמצעות תיקוף צולב (k-fold Cross-Validation) וזאת כדי לסווג את רמת המים הנוכחית לאחד מחמישה טווחים מוגדרים מראש (עד 20%, 20%-40%, 40%-60%, 60%-80%, ו-80%-100%). דוח זה מספק מסגרת עבודה מלאה ומבוססת נתונים, החל משלב איסוף הנתונים ועד להערכת הביצועים הסופית, ומשמש כהוכחת קונספט מובנית עבור מערכות חישה אקוסטיות שאינן פועלות בזמן אמת.

אופן ביצוע ההקלטות ומדידת המים

בשלב הראשון נבחרו שלושה סוגים שונים של כוסות. עבור כל כוס נמדד הנפח המקסימלי שהיא יכולה להכיל במיליטרים. לאחר מכן חולק נפח הכוס לחמישה טווחי מיליו מוגדרים לפי מיליטרים בטווחים הבאים: 0%-20%, 20%-40%, 40%-60%, 60%-80% ו-80%-100%.

עבור כל כוס חושב הנפח המתאים לכל אחד מהטווחים, כך שניתן לדעת באיזה שלב של המיליו נמצאת הכוס מבחינה כמותית. לאחר מכן בוצעו הקלטות אודיו של תהליך מזיגת המים לכל אחד מהטווחים במרחק זהה מזרם המים. עבור כל טווח מיליו הוקלטו 15 הקלטות נפרדות, במטרה ליצור מאגר נתונים מגוון ומספיק לאימון ובדיקת המודל.

ההקלטות בוצעו באמצעות מתקן תמי 4, שתדר הפעולה שלו הוא כ-260 Hz במהלך ניתוח האות נמצא כי לעיתים תדר זה גובר על רעש המים מבחינת העוצמה בספקטרום, ולכן עלול להשפיע על חילוץ המאפיינים מתוך ההקלטות. כדי לצמצם את ההשפעה של תדר הפעולה של המכשיר, הופעל מסנן High-Pass בתדר חיתוך של 270 Hz.

קטע קוד:

```
y, sr = librosa.load(audio_path, sr=sr)
duration = librosa.get_duration(y=y, sr=sr)

# Apply zero-phase Butterworth high-pass filter to remove frequencies below low_cut_hz
sos = sps.butter(hp_order, low_cut_hz, btype="highpass", fs=sr, output="sos")
y = sps.sosfiltfilt(sos, y)

results: dict = {
    "duration_s": round(duration, 4),
    "Cup": cup_num,
    "Fill": fill_label
}

# Calculate window size in samples
window_samples = len(y) // n_windows

# Precompute STFT for spectral features (use explicit n_fft and hop_length for consistent mapping)
n_fft = 2048
hop_length = 512
window_type = "hann"
S = librosa.stft(y, n_fft=n_fft, hop_length=hop_length, window=window_type)
S_abs = np.abs(S)
freqs = librosa.fft_frequencies(sr=sr, n_fft=n_fft)

# Zero out STFT bins below low_cut_hz (extra safety)
low_bin_mask = freqs < low_cut_hz
if np.any(low_bin_mask):
    S_abs[low_bin_mask, :] = 0.0
```

סך ההקלטות שנאספו:

$$3 \text{ cups} \times 5 \text{ ranges} \times 15 \text{ recordings} = 225 \text{ recordings}$$

כלומר, מאגר הנתונים הסופי כלל 225 הקלטות אודיו, כאשר כל הקלטה משויכת לסוג הכוס ולטווח מיליו המים המתאים. לאחר איסוף ההקלטות, בוצע עיבוד אותות אקוסטיים וחילוץ

מאפיינים מתוך כל הקלטה, ההקלטות הומרו לייצוג מספרי באמצעות חילוץ מאפיינים אקוסטיים, כך שניתן היה להשתמש בהן כקלט למודלי למידת מכונה.

בנוסף לסיווג רמת מילוי המים, חושב גם קצב זרימת המים עבור כל כוס. קצב הזרימה מתאר את כמות המים שנמזגת ביחידת זמן, ונמדד ביחידות של mL/s. חישוב זה מאפשר לבדוק האם קיימת עקביות בין תנאי ההקלטה השונים, וכן לבחון האם ניתן ללמוד מתוך האות האקוסטי מידע נוסף מעבר לסיווג טווח המילוי.

$$flow\ rate = \frac{volume}{time}$$

קטע קוד:

```
def compute_flow_rates(
    csv_path: str,
    test_indices: np.ndarray | list[int],
    tune: bool = True,
    cv: int = 5,
    random_state: int = 42,
    cup_sizes_ml: dict[int, float] = CUP_SIZES_ML,
    verbose: bool = True,
) -> dict[str, object]:
    test_indices = np.asarray(test_indices)
    X, y, fill_all, cup_all = _load_target_and_features(csv_path, cup_sizes_ml)

    # Same held-out test rows as the classifier; train on the rest.
    test_pos = np.sort(test_indices)
    train_pos = np.setdiff1d(np.arange(len(X)), test_pos)

    X_tr, X_te = X.iloc[train_pos], X.iloc[test_pos]
    y_tr, y_te = y[train_pos], y[test_pos]

    scaler = StandardScaler().fit(X_tr)
    X_tr_s, X_te_s = scaler.transform(X_tr), scaler.transform(X_te)

    cv_splitter = KFold(n_splits=cv, shuffle=True, random_state=random_state)

    candidates: dict[str, tuple] = {}
    for kind in ("xgb", "forest"):
        if tune:
            model, params, cv_r2 = _tune_regressor(kind, X_tr_s, y_tr, cv_splitter, random_state)
        else:
            model = (xgb.XGBRegressor(objective="reg:squarederror", n_estimators=300,
                                     max_depth=5, learning_rate=0.05, subsample=0.6,
                                     n_jobs=-1, verbosity=0, random_state=random_state)
                    if kind == "xgb"
                    else RandomForestRegressor(n_estimators=300, max_features="sqrt",
                                             n_jobs=-1, random_state=random_state))
            cv_r2 = float(cross_val_score(model, X_tr_s, y_tr, cv=cv_splitter,
                                         scoring="r2", n_jobs=-1).mean())
            params = {}
            candidates[kind] = (model, params, cv_r2)

    best_kind = max(candidates, key=lambda k: candidates[k][2])
    best_model, best_params, best_cv_r2 = candidates[best_kind]
    best_label = "XGBoost" if best_kind == "xgb" else "RandomForest"
```

```

best_model.fit(X_tr_s, y_tr)
pred = best_model.predict(X_te_s)

metrics = {
    "MAE": float(mean_absolute_error(y_te, pred)),
    "RMSE": float(np.sqrt(mean_squared_error(y_te, pred))),
    "R2": float(r2_score(y_te, pred)),
}

per_sample = pd.DataFrame({
    "csv_row": test_pos,
    "cup": cup_all[test_pos],
    "fill_pct": fill_all[test_pos],
    "true_flow_rate_mL_s": y_te,
    "pred_flow_rate_mL_s": pred,
})

per_class = (
    per_sample
    .groupby("fill_pct")[["true_flow_rate_mL_s", "pred_flow_rate_mL_s"]]
    .mean().reset_index().sort_values("fill_pct").reset_index(drop=True)
)

mean_true = float(np.mean(y_te))
mean_pred = float(np.mean(pred))

if verbose:
    print(f"\n{'=' * 64}")
    print("Flow-rate regression — best-suited model selection")
    print(f"{'=' * 64}")
    for kind, (_, params, cv_r2) in candidates.items():
        tag = " <-- chosen" if kind == best_kind else ""
        name = "XGBoost" if kind == "xgb" else "RandomForest"
        print(f"{name:<14} CV R2={cv_r2:.4f} params={params}{tag}")
    print(f"{'-' * 64}")
    print(f"Best model: {best_label}")
    print(f"Test-set: MAE={metrics['MAE']:.4f} "
          f"RMSE={metrics['RMSE']:.4f} R2={metrics['R2']:.4f} (mL/s)")
    print(f"{'-' * 64}")
    print(f"fill %:<10}{TRUE':>14}{PREDICTED':>14}")
    for _, r in per_class.iterrows():
        print(f"{r['fill_pct']:<10.0f}"
              f"{r['true_flow_rate_mL_s']:>14.4f}"
              f"{r['pred_flow_rate_mL_s']:>14.4f}")
    print(f"{'=' * 64}")

return {
    "mean_flow_rate_true": mean_true,
    "mean_flow_rate_predicted": mean_pred,
    "per_class": per_class,
    "per_sample": per_sample,
    "best_model": best_label,
    "best_params": best_params,
    "best_cv_r2": best_cv_r2,
    "metrics": metrics,
    "cv_scores": {k: v[2] for k, v in candidates.items()},
    "model": best_model,
    "scaler": scaler,
}

```

רקע תיאורטי

Feature selection (בחירת מאפיינים)

בתהליכי פיתוח של מודלים בלמידת מכונה קיימת חשיבות רבה לשלב בחירת המאפיינים. מטרת שלב זה היא לזהות את המאפיינים הרלוונטיים ביותר לבעיה, ולסנן מאפיינים שאינם תורמים למודל, מאפיינים כפולים, או מאפיינים המכילים מידע חופף.

בחירת מאפיינים נכונה עשויה לתרום להפחתת ממד הבעיה, לשיפור ביצועי המודל, להקטנת זמן החישוב, לשיפור יכולת ההכללה של המודל ולצמצום הסיכון לתופעת התאמת יתר (overfitting).

תופעת Overfitting מתרחשת כאשר מודל מציג שגיאת אימון נמוכה מאוד, ולעיתים אף אפסית, אך נכשל בביצוע חיזוי מדויק על נתונים חדשים שלא הופיעו בזמן האימון. במצב זה, המודל אינו לומד את החוקיות הכללית של המערכת, אלא מתאים את עצמו באופן מוגזם לרעש, לחריגות ולדפוסים מקריים הקיימים בסט האימון בלבד. תופעה זו עלולה להיגרם כתוצאה ממורכבות יתר של המודל, עודף מאפיינים, מספר גדול של דרגות חופש, או אימון ממושך מדי.

שימוש בקוד: כאן מופעל תהליך בחירת המאפיינים בפועל. הקוד משתמש ב- Forward Feature Selection לאחר דירוג ראשוני לפי ANOVA.

```
# Forward feature selection
selected_indices: list[int] | None = None
selected_features = features
if use_feature_selection:
    selected_indices, selected_features, _ = forward_feature_selection(
        X_train,
        y_train,
        features,
        model_params,
        build_model,
        max_features= max_features,
        cv=cv,
        initial_features= anova_results['feature'].head(n_initial).tolist()
    )
X_train = X_train.iloc[:, selected_indices]
X_test = X_test.iloc[:, selected_indices]
```


שיטות לבחירת מאפיינים:

1) שיטות סינון (filter methods)

שיטות סינון הן שיטות שאינן תלויות במודל הלמידה עצמו. הן מעריכות את התרומה של כל מאפיין על בסיס מדדים סטטיסטיים, עוד לפני שלב האימון של המודל. היתרון המרכזי של שיטות אלו הוא שהן פשוטות, מהירות יחסית, ואינן דורשות אימון חוזר של המודל עבור כל קבוצת מאפיינים.

שימוש בקוד: בקוד מיושם Filter Method מסוג ANOVA F-score שמדרג את המאפיינים לפי התרומה שלהם להפרדה בין מחלקות המילוי.

```
def anova_f_scores(X: pd.DataFrame, y_encoded) -> pd.DataFrame:
    if isinstance(y_encoded, np.ndarray):
        y_encoded = pd.Series(y_encoded)
    if len(y_encoded) != len(X):
        raise ValueError("X and y must have the same number of rows")

    # Select numeric feature columns
    numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()
    if not numeric_cols:
        raise ValueError("no numeric feature columns found to test")

    features, f_vals, p_vals = [], [], []
    for col in numeric_cols:
        xi_df = pd.concat([X[col], y_encoded], axis=1)
        xi_df.columns = [col, "target"]
        if xi_df.empty:
            continue
        xi_X = xi_df[col].values.reshape(-1, 1)
        xi_y = xi_df["target"]
        # encode target per-feature if still non-numeric (safeguard)
        if xi_y.dtype.kind not in "iuf":
            xi_y = LabelEncoder().fit_transform(xi_y.astype(str))
        f, p = f_classif(xi_X, xi_y)
        features.append(col)
        f_vals.append(float(f[0]))
        p_vals.append(float(p[0]))

    result = pd.DataFrame({"feature": features, "F": f_vals, "p": p_vals})
    result = result.sort_values("F", ascending=False).reset_index(drop=True)
    return result
```

```
def plot_anova_f_scores(
    scores_df: pd.DataFrame,
    top_n: Optional[int] = None,
    figsize: tuple = (10, 6),
    f_threshold: Optional[float] = None,
    palette: str = "viridis",
):
    if not {"feature", "F"}.issubset(scores_df.columns):
        raise ValueError("scores_df must contain 'feature', 'F' column")

    df = scores_df.copy().sort_values("F", ascending=False)
    if top_n is not None:
        df = df.head(top_n)
```

```

plt_style = sns.color_palette(palette, n_colors=len(df))
fig, ax = plt.subplots(figsize=figsize)
bars = ax.barh(df["feature"][::-1], df["F"][::-1], color=plt_style[::-1])
for bar, f in zip(bars, df["F"][::-1]):
    width = bar.get_width()
    ax.text(width + max(df["F"].max() * 0.01, 1e-6), bar.get_y() + bar.get_height() / 2,
            '{:.0f}'.format(f), va="center", fontsize=9)

ax.set_xlabel("ANOVA F-score")
ax.set_ylabel("Feature")
ax.set_title("ANOVA F-scores by Feature")
ax.grid(axis="x", linestyle="--", alpha=0.4)

if f_threshold is not None:
    ax.axvline(f_threshold, color="red", linestyle="--", linewidth=1)
    ax.text(f_threshold, -0.4, f"F = {f_threshold}", color="red", va="bottom", ha="right",
            fontsize=9)

plt.tight_layout()
plt.show()

```

α Variance threshold

שיטת Variance Threshold מבוססת על ההנחה שמאפיין שאינו משתנה באופן משמעותי בין הדגימות אינו מספק מידע מועיל למודל. אם למאפיין מסוים קיימת שונות נמוכה מאוד, משמעות הדבר היא שערכיו כמעט קבועים לאורך כל הדגימות, ולכן הוא כנראה אינו תורם ליכולת ההפרדה או החיזוי של המודל.

שיטה מסירה מאפיינים שהשונות שלהם נמוכה מסף מוגדר מראש:

$$\text{var}(x_j) < \theta$$

- x_j - המאפיין הנבדק
- θ - סף השונות

ב) MUTUAL INFORMATION

מדד Mutual Information בוחן את מידת התלות הסטטיסטית בין מאפיין מסוים לבין משתנה המטרה. בניגוד למדד מתאם לינארי רגיל, מידע הדדי מסוגל לזהות גם קשרים שאינם בהכרח לינאריים. באופן כללי, המדד בודק כמה מידע על משתנה המטרה ניתן לקבל מתוך ידיעת ערכו של מאפיין מסוים. ככל שהערך גבוה יותר, כך המאפיין נחשב רלוונטי יותר למשימת הסיווג או החיזוי. נוסחה הכללית:

$$I(x; y) = \sum_x \sum_y p(x, y) \log \left(\frac{p(x, y)}{p(x)p(y)} \right)$$

- x - מאפיין
- y - משתנה מטרה
- $p(x, y)$ - פונקציית הסתברות משותפת
- $p(x), p(y)$ - פונקציות הסתברות שוליות

2) שיטות עטיפה (wrapper methods)

שיטות עטיפה הן שיטות המשתמשות במודל הלמידה עצמו כדי להעריך קבוצות שונות של מאפיינים. בניגוד לשיטות סינון, כאן המאפיינים נבחרים לפי הביצועים בפועל של המודל לאחר אימון. מכיוון שבדיקה של כל הצירופים האפשריים של מאפיינים עלולה להיות יקרה מאוד מבחינה חישובית, נעשה לרוב שימוש באלגוריתמים חמדניים.

- אלגוריתם חמדני הוא אלגוריתם שבכל שלב בוחר את האפשרות שנראית הטובה ביותר באופן מקומי, מבלי לבדוק בהכרח את כל ההשלכות העתידיות של הבחירה.

forward selection (א)

בשיטת Forward Selection מתחילים מקבוצה ריקה של מאפיינים. בכל שלב נבדק איזה מאפיין, מבין המאפיינים שעדיין לא נבחרו, משפר בצורה המשמעותית ביותר את ביצועי המודל כאשר מוסיפים אותו לקבוצה הקיימת.

האלגוריתם ממשיך להוסיף מאפיינים באופן איטרטיבי עד שלא מתקבל שיפור משמעותי נוסף בביצועי המודל, או עד שמגיעים למספר מאפיינים שהוגדר מראש.

$$x_j = \arg \cdot \max_{x \in S_{(t-1)}} (\text{score}) \left(M(S_{(t-1)} \cup \{x\}) \right)$$

- s_t – קבוצת מאפיינים עד לשלב t
- x_j – מאפיין שעדיין לא נבחר

שימוש בקוד: בכל שלב נבדק איזה מאפיין מוסיף את השיפור הגבוה ביותר לביצועי המודל.

```
def forward_feature_selection(
    X_train: pd.DataFrame,
    y_train: pd.DataFrame,
    feature_names: list[str],
    model_params: dict[str, Any],
    build_model: Callable,
    random_state: int = 42,
    max_features: int | None = None,
    cv: int = 5,
    scoring: str = 'accuracy',
    initial_features: list[str] = [],
    min_improvement: float = 0, #1e-6,
    verbose: bool = True,
) -> tuple[list[int], list[str], list[dict[str, object]]]:
    n_features = X_train.shape[1]
    target = n_features if max_features is None else min(max_features,
n_features)
    cv_splitter = StratifiedKFold(n_splits=cv, shuffle=True,
random_state=random_state)

    selected: list[int] = [feature_names.index(feature) for feature in
initial_features]
    remaining: list[int] = [i for i in range(n_features) if i not in selected]
    #list(range(n_features))
    history: list[dict[str, object]] = []
    best_score = -np.inf

    if verbose:
        print(f"\nForward Feature Selection (wrapper, {cv}-fold CV,
scoring='{scoring}')"
        print(f"Candidate features: {n_features} | target N': {target}")

    model = build_model(**model_params, random_state = random_state)

    pipe = Pipeline([
        ("scaler", StandardScaler()),
        ("model", model)
    ])
```

```

while remaining and len(selected) < target:
    round_best_score = -np.inf
    round_best_feat: int | None = None

    # Try adding each remaining feature on top of the current set.
    for feat in remaining:
        trial = selected + [feat]
        scores = cross_val_score(
            pipe, X_train.iloc[:, trial], y_train,
            cv=cv_splitter, scoring=scoring, n_jobs=5,
        )
        mean_score = float(scores.mean())
        if mean_score > round_best_score:
            round_best_score = mean_score
            round_best_feat = feat

    # Stop if the best candidate this round doesn't improve enough.
    if round_best_feat is None or round_best_score <= best_score +
min_improvement:
        if verbose:
            print(f" No further improvement (best candidate "
                  f"{round_best_score:.4f} vs current {best_score:.4f}).
Stopping.")
        break

    selected.append(round_best_feat)
    remaining.remove(round_best_feat)
    best_score = round_best_score
    history.append({
        'feature': feature_names[round_best_feat],
        'index': round_best_feat,
        'cv_score': round_best_score,
    })
    if verbose:
        print(f" [ {len(selected)} ] + {feature_names[round_best_feat]:<25}
"
              f"CV {scoring}: {round_best_score:.4f}")

    selected_names = [feature_names[i] for i in selected]
    if verbose:
        print(f"Selected {len(selected)}/{n_features} features:
{selected_names}")

    return selected, selected_names, history

```

backward elimination (ב)

בשיטת Backward Elimination מתחילים מקבוצה הכוללת את כל המאפיינים. בכל שלב מסירים מאפיין אחד זה שהסרתו פוגעת בצורה המינימלית ביותר בביצועי המודל, או לחלופין משפרת אותם.

מתחילים מהקבוצה:

$$S_0 = \{x_1, x_2, \dots, x_N\}$$

בכל שלב מסירים את המאפיין הפחות משמעותי:

$$S_{t+1} = \frac{S_t}{\{x_j\}}$$

כאשר:

$$x_j = \arg(\min)_{(x_j \in S_t)} \Delta \text{score}$$

כלומר, המאפיין שההשפעה שלו על המודל היא הקטנה ביותר מוסר.

- S_0 – קבוצת המאפיינים ההתחלתית.
- $x_1, x_2, \dots, x_N$ – מאפיינים קיימים.
- N – מספר המאפיינים שיש בתהליך.
- S_t – קבוצת המאפיינים בשלב t .
- $\{x_j\}$ – המאפיין שאותו מסירים מהקבוצה.

3) שיטות מובנות (embedded methods)

שיטות מובנות הן שיטות שבהן בחירת המאפיינים מתבצעת כחלק אינטגרלי מתהליך האימון של המודל עצמו. כלומר, המודל לא רק לומד לבצע חיזוי או סיווג, אלא גם מספק אינדיקציה לגבי חשיבות המאפיינים.

דוגמה מרכזית לכך היא מודלים מבוססי עצים, כגון Random Forest ו-XGBoost. במודלים אלו ניתן להעריך את חשיבות המאפיינים לפי תרומתם לפיצולים בעץ ולשיפור פונקציית ההפסד.

במודל XGBoost אחד המדדים המרכזיים להערכת חשיבות מאפיין הוא מדד ה-Gain. מדד זה מתאר את השיפור בפונקציית ההפסד בעקבות פיצול של צומת בעץ על פי מאפיין מסוים.

נוסחת ה-Gain עבור פיצול ב-XGBoost היא:

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

- G_L, G_R – סכומי גרדיאנט בצדדים של הפיצול.
- H_L, H_R – סכומי הנגזרת השנייה.
- λ – פרמטר רגולציה.
- γ – קנס על יצירת פיצול בעץ.

שימוש בקוד: חילוץ חשיבות המאפיינים מתוך המודל עצמו.

```
def get_feature_importance(
    model: xgb.XGBClassifier | RandomForestClassifier,
    feature_names: list[str],
    top_n: int = 10
) -> None:
    importances = model.feature_importances_
    indices = np.argsort(importances)[-1:-top_n]

    print(f"\nTop {top_n} Important Features (Gain-based):")
    for rank, idx in enumerate(indices, 1):
        print(f"{rank}. {feature_names[idx]}: {importances[idx]:.4f}")
```

מודל XGBoost (eXtreme Gradient Boosting)

מודל XGBoost הוא אלגוריתם למידת מכונה מתקדם המבוסס על אנסמבל של עצי החלטה. המודל מתאים במיוחד לנתונים טבלאיים ומשמש הן למשימות סיווג והן למשימות רגרסיה. האלגוריתם פועל לפי עקרון של Gradient Boosting כלומר, המודל נבנה באופן סדרתי, כאשר כל עץ חדש מנסה לתקן את השגיאות של העצים הקודמים. בכל שלב האלגוריתם בוחן את השאריות או את כיוון השיפור של פונקציית ההפסד, ובונה עץ נוסף שמטרתו לצמצם את השגיאה הכוללת.

החיזוי הסופי של המודל מתקבל משילוב התרומות של כלל העצים:

$$\hat{y}_i = \sum_{k=1}^k f_k(x_i)$$

- f_k - מייצג את העץ
- K - מספר העצים הכולל

אחד היתרונות המרכזיים של XGBoost הוא שימוש ברגולריזציה, המפחיתה את הסיכון להתאמת יתר ומשפרת את יכולת ההכללה של המודל. בנוסף, האלגוריתם יעיל מבחינה חישובית, תומך בטיפול בערכים חסרים, ומאפשר להפיק מדדי חשיבות מאפיינים.

שימוש בקוד: הגדרת המודל XGBoost כולל הפרמטרים שלו.

```
def build_xgb(
    n_estimators: int = 100,
    learning_rate: float = 0.1,
    max_depth=6,
    random_state: int = 42,
    num_class: int | None = 5,
    verbosity: int = 0,
    subsample: float = 0.4,
    min_child_weight: int = 1,
    colsample_bytree: float = 0.55,
    objective: str = 'multi:softprob',
    n_jobs: int = 5,
    reg_alpha: float = 0.5,
    reg_lambda: float = 0.3
) -> xgb.XGBClassifier:

    return xgb.XGBClassifier(
        n_estimators=n_estimators,
        max_depth= max_depth,
        learning_rate= learning_rate,
        subsample= subsample,
        colsample_bytree= colsample_bytree,
        objective= objective,
        random_state=random_state,
        n_jobs=n_jobs,
        min_child_weight= min_child_weight,
        verbosity=verbosity,
        reg_alpha = reg_alpha,
        reg_lambda = reg_lambda,
        num_class = num_class
    )
```


Cross-Validation

על מנת לקבל אומדן מהימן ויציב יותר של ביצועי המודל על נתונים חדשים, נעשה שימוש בשיטת Cross-Validation ובפרט במבנה של K-Fold Cross-Validation.

בשיטה זו מחלקים את מאגר הנתונים ל-K תתי-קבוצות, הנקראות folds. בכל איטרציה, תתי-קבוצה אחת משמשת כסט בדיקה, בעוד שאר תתי-הקבוצות משמשות כסט אימון. התהליך חוזר על עצמו K פעמים כך שבכל פעם fold אחר משמש לבדיקה. בסיום התהליך מחשבים את ממוצע הביצועים מכל האיטרציות, וכך מתקבל מדד יציב ומייצג יותר לביצועי המודל.

שיטה זו חשובה משום שהיא מפחיתה את התלות בחלוקה מקרית אחת של הנתונים, מנצלת בצורה טובה יותר את כלל הדגימות, ומאפשרת הערכה אמינה יותר של יכולת ההכללה של המודל.

עם זאת, חשוב להדגיש כי Cross-Validation אינה מוכיחה שהמודל אינו סובל מ-Overfitting. היא רק מאפשרת לזהות בצורה טובה יותר האם קיימת בעיית הכללה. בנוסף, כדי למנוע Data Leakage, יש לבצע פעולות כמו נרמול, בחירת מאפיינים ואימון המודל בתוך כל fold בנפרד, ולא על כל בסיס הנתונים לפני החלוקה.

שימוש בקוד: הנרמול מתבצע בתוך כל Fold בנפרד, כאשר ה-Scaler מתאים את עצמו רק לנתוני האימון ואז מוחל על נתוני הבדיקה.

```
X_train_scaled, X_val_scaled, _ = scale_features(X_train, X_val)
```

```
steps: list[tuple[str, Any]] = [("scaler", StandardScaler())]
if select_k is not None:
    steps.append(("select", SelectKBest(f_classif, k=min(select_k, X_raw.shape[1])))
steps.append(("model", build_model(**model_params, random_state=random_state)))
pipe = Pipeline(steps)

folds = make_cup_stratified_folds(cup_labels, fill_labels, n_splits,
                                  n_repeats, random_state)

scorers = _resolve(list(scoring) + ["mae_classes"])
cv_out = cross_validate(pipe, X_raw, y, cv=folds, scoring=scorers,
                        n_jobs=-1, error_score="raise")
```

בפרויקט זה נרמול הנתונים בוצע בתוך תהליך ה-K-Fold, בנפרד עבור כל Fold. כלומר, בכל חלוקה הנרמול חושב על בסיס נתוני האימון בלבד, ולאחר מכן הוחל על נתוני הבדיקה של אותו Fold. שיטה זו מונעת זליגת מידע מנתוני הבדיקה אל שלב האימון, ומבטיחה הערכה אמינה יותר של ביצועי המודל.

במקרה של בעיית סיווג, מומלץ להשתמש ב-Stratified K-Fold כך שההתפלגות של המחלקות תישמר באופן דומה בכל אחד מה-folds.

מדוע אנו משתמשים בשיטה זו?

השימוש ב-K-Fold Cross-Validation מאפשר לנצל את מאגר הנתונים בצורה יעילה יותר. כל דגימה משתתפת גם באימון וגם בבדיקה, אך במחזורים שונים. כך מתקבלת הערכה מאוזנת יותר של ביצועי המודל.

בנוסף, השיטה מאפשרת לבדוק האם המודל יציב, כלומר האם הביצועים שלו נשארים דומים גם כאשר משנים את חלוקת הנתונים. אם הביצועים משתנים בצורה משמעותית בין folds שונים, הדבר עשוי להעיד על רגישות גבוהה של המודל או על בעיית הכללה.

שימוש בקוד: מתבצע תיקוף צולב ובו נרמול בתוך כל fold במטרה למנוע זליגת מידע.

```
def cross_validate_model(
    model: xgb.XGBClassifier | RandomForestClassifier,
    X: pd.DataFrame,
    y: np.ndarray,
    random_state: int = 42,
    cv: int = 5,
    scoring: tuple[str, ...] = ('accuracy', 'f1_macro', 'precision_macro', 'recall_macro'),
    verbose: bool = True,
) -> dict[str, object]:
    cv_splitter = StratifiedKFold(n_splits=cv, shuffle=True, random_state=random_state)

    if verbose:
        print(f"\n{'='*60}")
        print(f"Stratified {cv}-Fold Cross-Validation (per-fold scaling)")
        print(f"{'='*60}")

    scorers = {metric: get_scorer(metric) for metric in scoring}
    fold_scores: dict[str, list[float]] = {metric: [] for metric in scoring}

    # StratifiedKFold.split returns positional indices, so use .iloc
    for fold_idx, (train_idx, val_idx) in enumerate(cv_splitter.split(X, y), 1):
        X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
        y_train, y_val = y[train_idx], y[val_idx]

        # Fit scaler on the training fold only, apply to both splits
        X_train_scaled, X_val_scaled, _ = scale_features(X_train, X_val)

        fold_model = clone(model)
        fold_model.fit(X_train_scaled, y_train)

        for metric in scoring:
            score = scorers[metric](fold_model, X_val_scaled, y_val)
            fold_scores[metric].append(score)

        if verbose:
            fold_summary = ", ".join(
                f"{metric}={fold_scores[metric][-1]:.4f}" for metric in scoring
            )
            print(f"Fold {fold_idx}/{cv}: {fold_summary}")

    results = {}
    for metric in scoring:
        scores = np.array(fold_scores[metric])
        results[metric] = {
            'scores': scores,
            'mean': float(scores.mean()),
            'std': float(scores.std()),
        }
    if verbose:
        fold_str = ", ".join(f"s: {s:.4f}" for s in scores)
        print(f"{metric}<18>: {scores.mean():.4f} +/- {scores.std():.4f} "
```

```

        f"(folds: [{fold_str}])")

if verbose:
    print(f"{'='*60}")

return results

```

```

def cross_validate_clean(
    build_model: Callable,
    model_params: dict[str, Any],
    X_raw,
    y,
    cup_labels,
    fill_labels,
    select_k: int | None = None,
    n_splits: int = 4,
    n_repeats: int = 5,
    random_state: int = 42,
    scoring: tuple[str, ...] = ("accuracy", "f1_macro",
                                "quadratic_kappa", "adjacent_acc"),
    verbose: bool = True,
) -> dict[str, dict[str, object]]:
    X_raw = np.asarray(X_raw); y = np.asarray(y)

    steps: list[tuple[str, Any]] = [("scaler", StandardScaler())]
    if select_k is not None:
        steps.append(("select", SelectKBest(f_classif, k=min(select_k, X_raw.shape[1])))
    steps.append(("model", build_model(**model_params, random_state=random_state)))
    pipe = Pipeline(steps)
    folds = make_cup_stratified_folds(cup_labels, fill_labels, n_splits,
                                      n_repeats, random_state)

    scorers = _resolve(list(scoring) + ["mae_classes"])
    cv_out = cross_validate(pipe, X_raw, y, cv=folds, scoring=scorers,
                            n_jobs=-1, error_score="raise")

    results = {}
    for metric in scoring:
        # charted metrics only
        s = cv_out[f"test_{metric}"]
        results[metric] = {"scores": s, "mean": float(s.mean()), "std": float(s.std())}
    mae = -cv_out["test_mae_classes"] # negate (greater_is_better=False)

    if verbose:
        sel = f", ANOVA top-{select_k}" if select_k else ""
        print(f"\n{'='*70}")
        print(f"Leak-free CV | (cup x fill)-stratified | "
              f"{n_splits}-fold x {n_repeats} repeats{sel}")
        print(f"{'='*70}")
        for metric in scoring:
            s = np.asarray(results[metric]["scores"])
            print(f"{metric}<18}: {s.mean():.4f} +/- {s.std():.4f} "
                  f"(min={s.min():.4f}, max={s.max():.4f})")
            print(f"{'mae_classes':<18}: {mae.mean():.4f} +/- {mae.std():.4f} "
                  f"(lower is better; avg fill-buckets off)")
        print(f"{'='*70}")
    return results

```

MFCC — Mel-Frequency Cepstral Coefficients

MFCC היא שיטה שמטרתה לחלץ מאפיינים מאותות אקוסטיים ולייצג את מבנה התדרים של האות בצורה קומפקטית ומשמעותית, כך שניתן יהיה להשתמש במקדמים שהתקבלו כקלט למודל למידת מכונה.

בשלב הראשון מחלקים את אות האודיו לחלונות זמן קצרים, ועל כל חלון מבצעים התמרת פורייה כדי לעבור ממרחב הזמן למרחב התדר. לאחר מכן מפעילים סדרת מסננים בסקאלת Mel ומחשבים את האנרגיה בכל מסנן, ולבסוף מפיקים את מקדמי ה-MFCC.

לאחר חישוב האנרגיות במסנני Mel מתקבלים מקדמי MFCC בעזרת טרנספורמציה נוספת שמטרתה לדחוס את המידע ולייצג את צורת הספקטרום במספר קטן של ערכים.

בשלב חילוץ המאפיינים הופקו מכל הקלטה מספר מאפיינים אקוסטיים המתארים את מבנה האות. בין המאפיינים שחולצו ניתן למנות MFCC, RMS Energy, Spectral Flatness, Dominant Frequency ו-Onset Count. מאפיינים אלו נבחרו משום שהם מתארים היבטים שונים של הצליל: עוצמת האות, מבנה התדרים, שינויי אנרגיה, תדירות דומיננטית ואירועים חדים באות. מכיוון שצליל מזיגת המים משתנה בהתאם לגובה המים בכוס, מאפיינים אלו מאפשרים למודל לקשר בין האות האקוסטי לבין טווח המילוי.

שימוש בקוד: חילוץ מקדמי MFCC מתוך אות האודיו. בגרסה לפי חלונות זמן נשמרים מקדמי MFCC לכל חלון בנפרד.

```
mfcc = librosa.feature.mfcc(
    y=y, sr=sr, n_mfcc=n_mfcc, n_fft=n_fft, hop_length=hop_length
)
# shape: (n_mfcc, frames)
for coeff_idx in range(n_mfcc):
    _add_stats(row, f"mfcc_{coeff_idx}", mfcc[coeff_idx])

# MFCC delta (velocity) and delta-delta (acceleration)
delta_mfcc = librosa.feature.delta(mfcc)
delta2_mfcc = librosa.feature.delta(mfcc, order=2)
for coeff_idx in range(n_mfcc):
    _add_stats(row, f"mfcc_delta_{coeff_idx}", delta_mfcc[coeff_idx])
    _add_stats(row, f"mfcc_delta2_{coeff_idx}", delta2_mfcc[coeff_idx])
```

```
# === Original Features (note: spectral features computed on filtered audio) ===
spectral_centroid = np.mean(librosa.feature.spectral_centroid(y=window_audio, sr=sr,
    window=window_type))
spectral_rolloff = np.mean(librosa.feature.spectral_rolloff(y=window_audio, sr=sr,
    window=window_type))
zero_crossing_rate = np.mean(librosa.feature.zero_crossing_rate(window_audio))
mfcc = np.mean(librosa.feature.mfcc(y=window_audio, sr=sr, n_mfcc=13), axis=1)
rms_energy = np.mean(librosa.feature.rms(y=window_audio)[0])
spectral_bandwidth = np.mean(librosa.feature.spectral_bandwidth(y=window_audio, sr=sr,
    window=window_type))
```

```
for index, coeff in enumerate(mfcc.tolist()):
    results[f"MFCC_{index}_window_{i}"] = float(coeff)
```

Hyperparameter Tuning

Hyperparameter Tuning הוא תהליך שבו בוחרים את הערכים המתאימים ביותר להיפר-פרמטרים של המודל. היפר-פרמטרים הם ערכים שנקבעים מראש לפני האימון, ואינם נלמדים ישירות מתוך הנתונים.

במודלים כמו XGBoost ניתן לכוון פרמטרים כגון מספר העצים, עומק העץ, קצב הלמידה ופרמטרי רגולריזציה. בחירה נכונה של ערכים אלו משפיעה על ביצועי המודל, על רמת הדיוק שלו ועל היכולת שלו להכליל לנתונים חדשים.

מטרת התהליך היא למצוא איזון בין מודל פשוט מדי, שעלול לא ללמוד מספיק את הנתונים, לבין מודל מורכב מדי, שעלול לגרום ל-Overfitting.

שימוש בקוד: כוון היפר-פרמטרים באמצעות RandomizedSearchCV.

```
def tune_hyperparameters(
    model: xgb.XGBClassifier | RandomForestClassifier,
    X: np.ndarray,
    y: np.ndarray,
    param_grid: dict[str, list] | None = None,
    cv: int = 5,
    scoring: str = 'accuracy',
    random_state: int = 42,
    n_iters: int = 100,
    verbose: bool = True,
) -> tuple[dict[str, int | float], float]:
    if param_grid is None:
        param_grid = {
            'n_estimators': [100, 200, 300, 400],
            'max_depth': [4, 6, 8, 10],
            'learning_rate': [0.05, 0.1],
            'subsample': [0.4, 0.55, 0.6, 0.7, 1.0],
        }

    cv_splitter = StratifiedKFold(n_splits=cv, shuffle=True, random_state=random_state)
    #base = build_xgb(random_state=random_state, num_class=num_class)

    if verbose:
        n_combos = int(np.prod([len(v) for v in param_grid.values()]))
        print(f"\n{'='*60}")
        print(f"Hyperparameter Tuning (grid search, {cv}-fold CV, "
              f"scoring='{scoring}')"
              f"Searching {n_combos} combinations x {cv} folds "
              f"={n_combos * cv} fits")
        print(f"{'='*60}")

    pipe = Pipeline([
        ("scaler", StandardScaler()),
        ("model", model)
    ])

    pipe_param_grid = {
        (key if key.startswith("model__") else f"model__{key}"): values
        for key, values in param_grid.items()
    }

    search = RandomizedSearchCV(
        estimator=pipe,
```

```

    param_distributions=pipe_param_grid,
    scoring=scoring,
    cv=cv_splitter,
    n_iter= n_iters,
    n_jobs= 3,
    random_state=random_state
)
search.fit(X, y)

best_params = best_params = {
    key.replace("model__", "", 1): value
    for key, value in search.best_params_.items()
}
best_score = float(search.best_score_)

if verbose:
    print(f"Best CV {scoring}: {best_score:.4f}")
    print(f"Best params: {best_params}")
    print(f"{'='*60}")

return best_params, best_score

```

```

best_params: dict[str, int | float] | None = None
best_tune_score: float | None = None
if tune:
    best_params, best_tune_score = tune_hyperparameters(
        model,
        X_train[selected_features],
        y_train,
        param_grid=tune_grid,
        n_iters=100,
        cv=tune_cv,
    )
    model_options.update(best_params)
    model = build_model(**model_options, random_state= random_state)

```

Random Forest

Random Forest הוא אלגוריתם למידת מכונה המבוסס על אוסף של עצי החלטה. במקום להסתמך על עץ החלטה יחיד, המודל בונה מספר רב של עצים, כאשר כל עץ מאומן על חלק שונה מהנתונים ועל חלק מהמאפיינים. שילוב זה מאפשר לקבל מודל יציב יותר ופחות רגיש לרעש או לשינויים קטנים בנתוני האימון.

בבעיית סיווג, כל עץ במודל נותן תחזית למחלקה מסוימת, והתחזית הסופית מתקבלת לפי הצבעת רוב בין כלל העצים. ניתן לתאר זאת באופן כללי כך:

$$\hat{y} = \text{majority vote } \{h_1(x), h_2(x), \dots, h_T(x)\}$$

- $h_T(x)$ – עץ החלטה בודד
- T – מספר עצים במודל.

היתרון המרכזי של Random Forest הוא הפחתת הסיכון ל- Overfitting ביחס לעץ החלטה יחיד, שיפור יציבות המודל, ויכולת להתמודד עם נתונים מורכבים ורועשים.

שימוש בקוד: הגדרת המודל random forest ושימוש.

```
def build_randomforest(
    n_estimators: int = 100,
    max_depth: int = 6,
    random_state: int = 42,
    n_jobs: int = 3,
    min_samples_split: int = 2,
    min_samples_leaf: int = 1,
    class_weight: str | None = None,
    max_features: str | None = 'log2',
    verbosity: int = 0,
) -> RandomForestClassifier:
    "Builds an untraineds random forest model"
    model_Forest = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        max_features=max_features,
        random_state=random_state,
        verbose=verbosity,
        n_jobs=n_jobs,
        min_samples_split=min_samples_split,
        min_samples_leaf=min_samples_leaf,
        class_weight=class_weight
    )
    return model_Forest
```

```
forest_options = {
    "n_estimators": 200,
    #"criterion": "gini",
    "max_depth": 8,
    "min_samples_split": 2,
    "min_samples_leaf": 1,
    #"min_weight_fraction_leaf": 0.0,
    "max_features": "log2",
```

```

# "max_leaf_nodes": None,
# "min_impurity_decrease": 0.0,
# "bootstrap": True,
# "oob_score": False,
# "n_jobs": 3,
# "verbosity": 0,
# "warm_start": False,
# "class_weight": None,
# "ccp_alpha": 0.0,
# "max_samples": None,
}

forest_tune_grid = {
    'n_estimators': [100, 200, 300, 400],
    'max_depth': [6, 8, 10,],
    'max_features': ['sqrt', 'log2'],
}

print('\n')
print('='*60)
print("Random Forest")
print('='*60)
results_Forest = model_pipeline(
    X, y, features, anova_results,
    forest_options, build_randomforest, model_type="FOREST",
    random_state= 42,
    use_feature_selection= True, max_features=100, n_initial= 46,
    run_cross_validation = True, cv = 4,
    tune = True, tune_grid= forest_tune_grid, tune_cv= 4,
    feature_impact_graph= False
)

graph_CM(results_Forest['confusion_matrix'], encoder.classes_.astype('str'), 'RandomForest')

```


Stratified Group K-Fold

Stratified Group K-Fold היא שיטת חלוקה מתקדמת המשמשת בתהליך ה-Cross-Validation. מטרת השיטה היא לחלק את הנתונים למספר folds תוך שמירה על שני עקרונות חשובים: שמירה על התפלגות המחלקות, ומניעת ערבוב של דגימות השייכות לאותה קבוצה בין סט האימון לסט הבדיקה.

בניגוד ל-K-Fold רגיל, שבו החלוקה מתבצעת באופן כללי בלבד, בשיטה זו נשמר היחס בין המחלקות השונות בכל fold.

ניתן לתאר את מטרת השיטה כך:

$$\text{Train groups} \cap \text{Test groups} = \emptyset$$

כלומר, אין חפיפה בין הקבוצות שנמצאות באימון לבין הקבוצות שנמצאות בבדיקה.

השיטה מסייעת לקבל הערכה יציבה ומציאותית יותר של ביצועי המודל, במיוחד כאשר קיימות מספר דגימות הקשורות לאותו מקור נתונים.

שילוב המודלים XGBoost ו-Random Forest

בפרויקט ביצענו שילוב של שני המודלים: XGBoost ו-Random Forest. כל אחד מהמודלים מפיק עבור כל הקלטה וקטור הסתברויות המתאר את הסיכוי שההקלטה שייכת לכל אחד מטווחי המחלקות של רמת מילוי המים.

לאחר מכן, מתבצע שילוב בין שתי ההסתברויות של שני המודלים באמצעות ממוצע משוקלל:

$$P_{final}(c | x) = \alpha \cdot P_{RF}(c | x) + \beta \cdot P_{XGB}(c | x)$$

- $P_{final}(c | x)$ – ההסתברות הסופית שהדגימה שייכת למחלקה C.
- $P_{RF}(c | x)$ – ההסתברות שהתקבלה ממודל Random Forest עבור מחלקה C.
- $P_{XGB}(c | x)$ – ההסתברות שהתקבלה ממודל XGBoost עבור מחלקה C.
- $\alpha = 0.6, \beta = 0.4$ – קבועי משקל הקובעים את התרומה היחסית של כל מודל לתחזית הסופית.

השילוב מאפשר לנצל את היתרונות של שני המודלים יחד. Random Forest מציג יציבות גבוהה יותר בין ה־ folds בעוד XGBoost מספק מנגנון למידה חזק המבוסס על תיקון שגיאות הדרגתי. לכן, שילוב ההסתברויות מאפשר לקבל החלטה סופית מאוזנת וחזקה יותר מאשר הסתמכות על מודל יחיד בלבד.

מטרת השילוב היא לנצל את היתרונות של שני המודלים על מנת לשפר את היציבות, להפחית טעויות נקודתיות ולחזק את יכולת ההכללה של המערכת עבור הקלטות נוספות.

קטע קוד:

```
def get_combined_prediction_results(
    XGB_CONST:float, probs_XGB: np.ndarray,
    FOREST_CONST:float, probs_Forest: np.ndarray,
    y_test: np.ndarray,
    label_encoder: LabelEncoder
    )-> dict[str, object]:
    """
    Combines both model probabilities by weighted sum and evaluating
    """
    probs = XGB_CONST * probs_XGB + FOREST_CONST * probs_Forest
    results = probs.argmax(axis = 1)

    # Convert numeric labels back to original class names
    class_names = label_encoder.classes_.astype('str')

    accuracy, cm = report_on_results(y_test, results, class_names)

    return {
        'accuracy': accuracy,
        'predictions': results,
        'confusion_matrix': cm,
    }
```

```
print('\n')
print('='*60)
print("Combined")
print('='*60)
XGB_CONST = 0.4
FORSET_CONST = 0.6
final_results = get_combined_prediction_results(
    XGB_CONST, results_XGB['probabilities'],
    FORSET_CONST, results_Forest['probabilities'],
    y_test, encoder)

graph CM(final_results['confusion matrix'], encoder.classes_.astype('str'), 'Combined')
```

מבנה הקוד:

לצורך ארגון נוח וברור של תהליך העבודה, הקוד חולק לשמונה קבצים מרכזיים, כאשר כל קובץ אחראי על שלב אחר בתהליך פיתוח המערכת:

(1 `recording_to_dataset` - ניתוח ספקטרלי ויצירת מאגר הנתונים
קובץ זה אחראי על טעינת קובצי האודיו הגולמיים, ביצוע ניתוח ספקטרלי וחילוך מאפיינים מתוך ההקלטות. הפלט של שלב זה הוא מאגר נתונים מסודר הכולל מאפיינים אקוסטיים לצד התוויות המתאימות של רמת מילוי המים.

(2 `data_load` - טעינת הנתונים ועיבוד מקדים
קובץ זה מטפל בהכנת הנתונים לאימון המודל. במסגרת שלב זה מתבצעים סינון עמודות שאינן רלוונטיות, חלוקה לנתוני אימון ובדיקה, ונרמול הנתונים. מטרת שלב זה היא לוודא שהמידע מוזן למודל בצורה תקינה ואחידה.

(3 `model_builder` -בנייה ואימון המודל
קובץ זה אחראי על הגדרת מודל הלמידה ואימון המודל על בסיס נתוני האימון. בשלב זה נקבעים מאפייני המודל, מתבצע תהליך הלמידה, ובמידת הצורך מתבצע גם כוונן הייפר-פרמטרים.

(4 `model_test` -בדיקת ביצועי המודל
קובץ זה בודק את ביצועי המודל לאחר האימון על נתוני בדיקה שלא שימשו בתהליך הלמידה. מטרתו להעריך את יכולת המודל לסווג נכון דגימות חדשות ולחשב מדדי ביצוע סופיים.

(5 `model_validation` -תיקוף המודל
קובץ זה מבצע Cross-Validation לצורך בדיקת יציבות המודל. התהליך מאפשר להעריך האם ביצועי המודל נשמרים גם כאשר חלוקת הנתונים משתנה, ובכך מסייע בזיהוי בעיות כמו Overfitting.

(6 `graphs_generator` -הפקת גרפים
קובץ זה אחראי על יצירת הגרפים המוצגים בפרויקט, כגון גרפי ביצועים, השוואה בין ערכים אמיתיים לחזויים, וחשיבות מאפיינים. ריכוז הגרפים בקובץ נפרד מאפשר שמירה על קוד מסודר וברור.

(7 `pipeline` -ניהול התהליך המרכזי
זהו הקובץ הראשי של הפרויקט. הוא מפעיל את כל שלבי העבודה לפי הסדר: טעינת הנתונים, עיבוד מקדים, אימון המודל, בדיקה, תיקוף והפקת תוצרים. בפועל, הרצת קובץ זה מפעילה את התהליך כולו מקצה לקצה.

(8 `model_saving` -שמירה וטעינה של המודל
קובץ זה אחראי על שמירת המודל המאומן לצורך שימוש חוזר בעתיד, ללא צורך באימון מחדש. בנוסף, הוא מאפשר לטעון מודל קיים ולהשתמש בו לביצוע חיזויים חדשים.

מדדי הערכה

לצורך הערכת ביצועי המודלים נעשה שימוש בארבעה מדדי הערכה.

מדד Accuracy מתאר את אחוז הסיווגים הנכונים מתוך כלל הדגימות.

$$Accuracy = \frac{\text{Number of correct predictions}}{\text{Total number of predictions}}$$

$$Accuracy = \frac{\sum C_{ii}}{\sum C_{ij}}$$

- C_{ii} – מספר הדגימות שסווגו נכון במחלקה i .
- C_{ij} – כלל הדגימות במטריצת בלבול.

מדד Precision בוחן מתוך כל הדגימות שהמודל סיווג למחלקה מסוימת, כמה אכן שייכות אליה.

$$Precision = \frac{TP}{(TP + FP)}$$

- TP – דגימות שסווגו נכון למחלקה.
- FP – דגימות שסווגו בטעות למחלקה, למרות שאינן שייכות.

מדד Recall בוחן מתוך כל הדגימות ששייכות למחלקה מסוימת, כמה מהן זוהו נכון על ידי המודל.

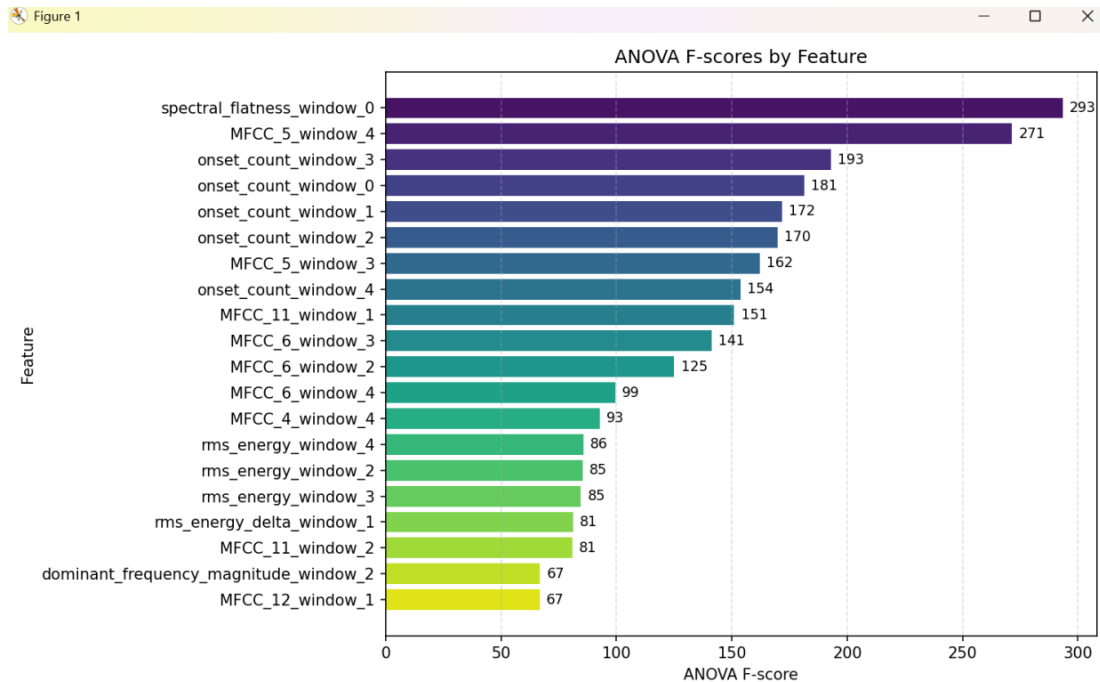
$$Recall = \frac{TP}{(TP + FN)}$$

- FN – דגימות ששייכות למחלקה אך המודל לא סיווג אותן אליה.

מדד F1-score משלב בין Precision ו- Recall, ולכן מתאים במיוחד כאשר רוצים לקבל מדד מאוזן יותר לביצועי הסיווג.

$$F1 - score = 2 \cdot \frac{Precision \cdot Recall}{(Precision + Recall)}$$

תוצאות הקוד:

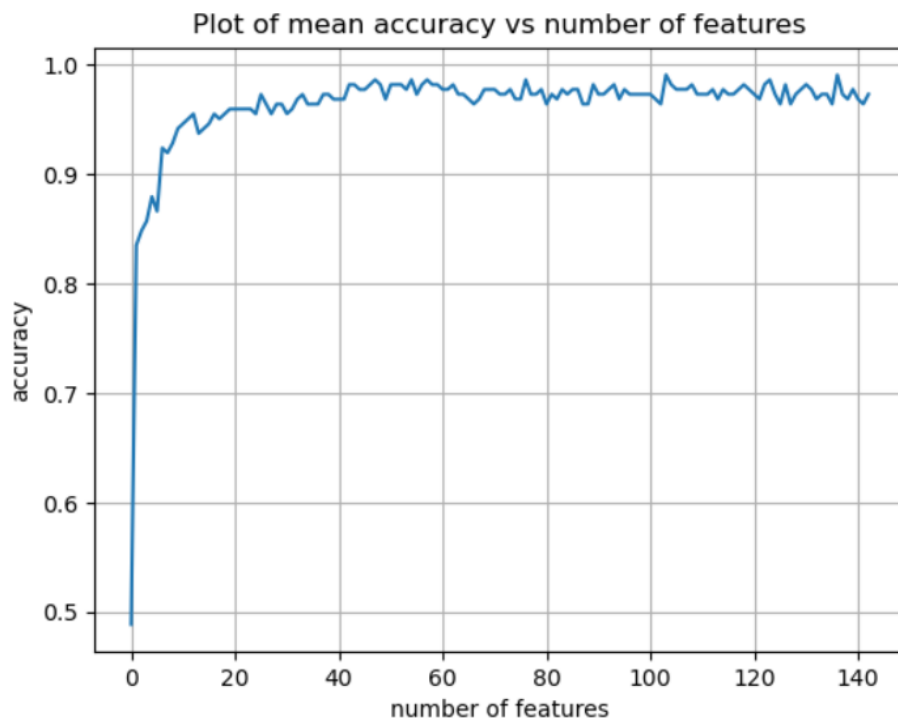


איור 1: דירוג המאפיינים האקוסטיים לפי ANOVA F-score

הגרף מציג את דירוג המאפיינים האקוסטיים לפי ANOVA F-score, כאשר ערך גבוה יותר מעיד על תרומה גדולה יותר לסיווג רמת המילוי.

ניתן לראות כי spectral_flatness_window_0 ו-MFCC_5_window_4 מציגים את הערכים הגבוהים ביותר, ולכן הם נחשבים למאפיינים משמעותיים במיוחד. בנוסף, מופיעים בדירוג גם מאפייני MFCC, RMS Energy, onset_count, ו-Dominant Frequency, דבר שמראה כי הסיווג מתבסס על שילוב של מבנה התדרים, עוצמת האות ושינויים בזמן.

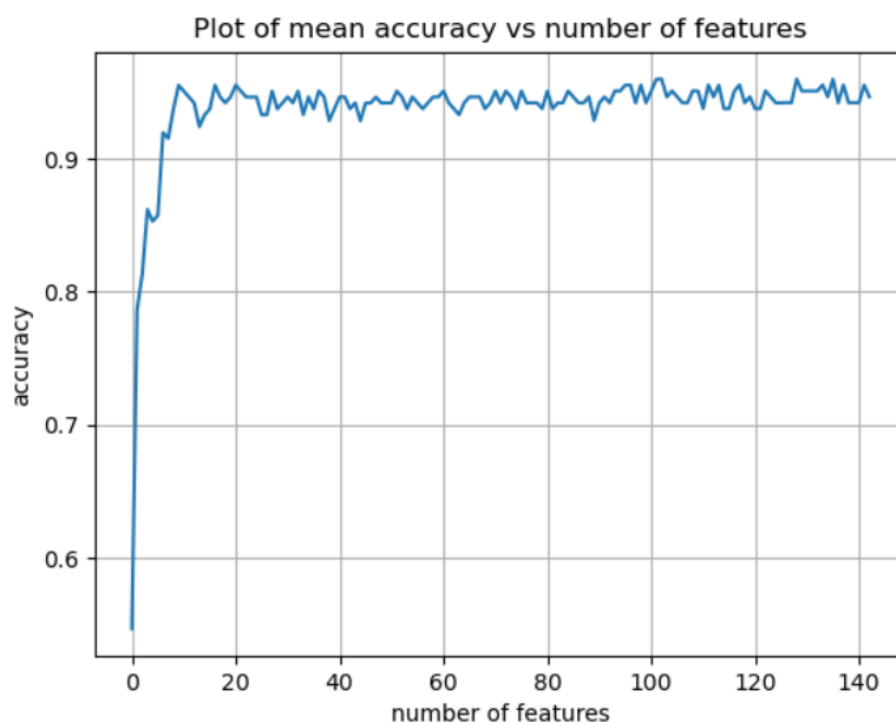
תוצאה זו מחזקת את הצורך בבחירת מאפיינים, כדי להשתמש רק במאפיינים התורמים ביותר למודל.



איור 2: השפעת מספר המאפיינים על דיוק מודל Random Forest

הגרף מציג את דיוק מודל Random Forest כתלות במספר המאפיינים שנבחרו. ניתן לראות כי כאשר מספר המאפיינים קטן מאוד, דיוק המודל נמוך יחסית. עם הוספת מאפיינים נוספים מתקבלת עלייה חדה בדיוק, עד להתייצבות סביב ערכים גבוהים של כ-97%-98%.

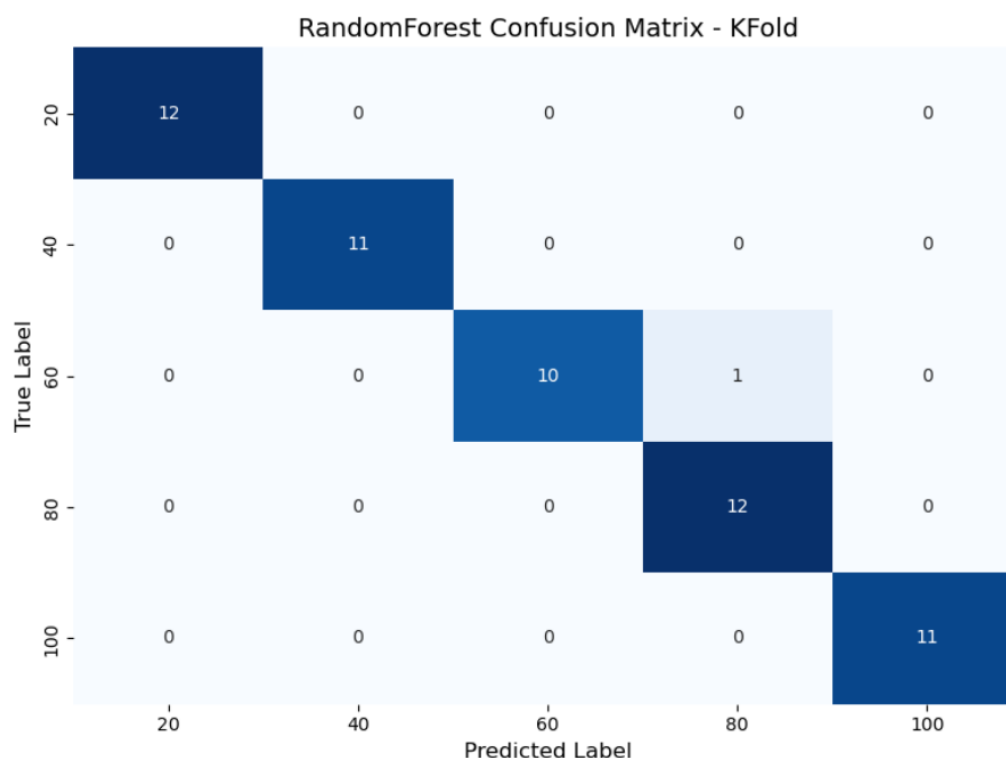
לאחר מספר מסוים של מאפיינים, הוספת מאפיינים נוספים אינה משפרת באופן משמעותי את ביצועי המודל. תוצאה זו מעידה כי קיימת קבוצת מאפיינים מרכזית המכילה את רוב המידע הדרוש לסיווג רמת מילוי המים.



איור 3: השפעת מספר המאפיינים על דיוק מודל XGBoost

הגרף מציג את דיוק מודל XGBoost כתלות במספר המאפיינים שנבחרו. ניתן לראות כי כאשר מספר המאפיינים קטן, דיוק המודל נמוך יותר. עם הוספת מאפיינים נוספים מתקבלת עלייה מהירה בביצועים, ולאחר מכן הדיוק מתייצב סביב ערכים גבוהים של כ-94%-96%.

משמעות הדבר היא שמודל XGBoost מצליח ללמוד את ההפרדה בין מחלקות המילוי גם בעזרת מספר מצומצם יחסית של מאפיינים. עם זאת, בהשוואה ל-Random Forest ניתן לראות כי Random Forest מציג ביצועים מעט גבוהים ויציבים יותר.

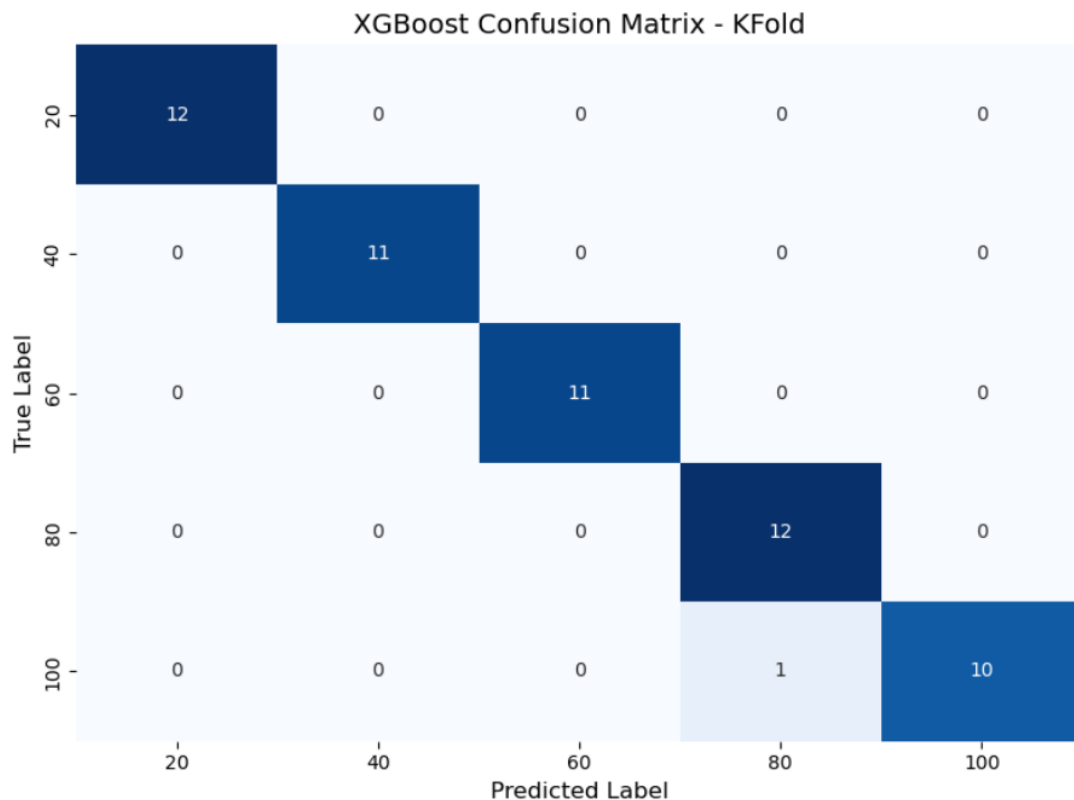


איור 4: מטריצת בלבול עבור מודל Random Forest בתהליך K-Fold

הגרף מציג מטריצת בלבול עבור מודל Random Forest לאחר תהליך K-Fold Cross-Validation. בציר האנכי מוצגות המחלקות האמיתיות של רמת מילוי המים, ובציר האופקי מוצגות המחלקות שחזה המודל.

ניתן לראות כי רוב הערכים נמצאים על האלכסון הראשי של המטריצה, כלומר רוב הדגימות סווגו למחלקה הנכונה. המודל סיווג באופן מלא את המחלקות 20%, 40%, 80% ו-100%. במחלקת 60% קיימת טעות אחת בלבד, שבה דגימה אחת סווגה בטעות למחלקת 80%.

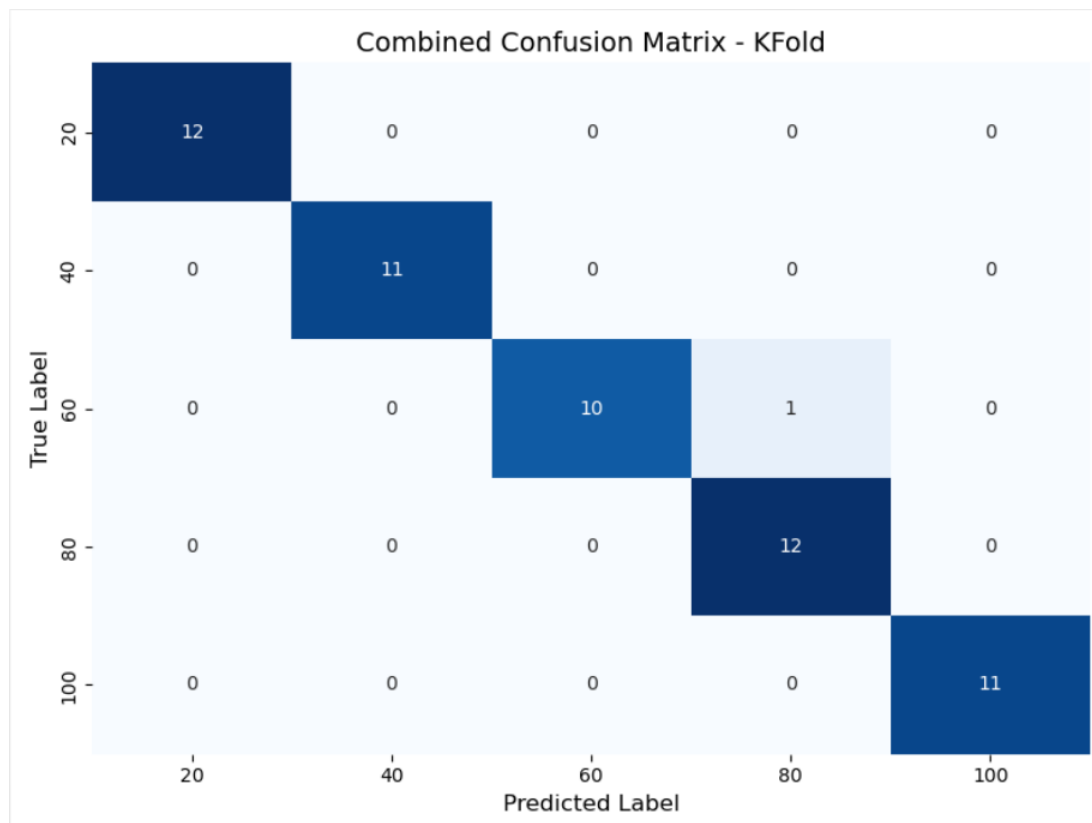
תוצאה זו מעידה על ביצועים גבוהים מאוד של מודל Random Forest ועל יכולת טובה להבחין בין טווחי מילוי המים השונים.



איור 5: מטריצת בלבול עבור מודל XGBoost בתהליך K-Fold

הגרף מציג מטריצת בלבול עבור מודל XGBoost לאחר תהליך K-Fold Cross Validation. ניתן לראות כי רוב הדגימות נמצאות על האלכסון הראשי, כלומר סווגו בצורה נכונה. המודל סיווג באופן מלא את המחלקות 20%, 40%, 60% ו-80%. במחלקת 100% קיימת טעות אחת בלבד, שבה דגימה אחת סווגה בטעות כמחלקת 80%. כלומר, עיקר הטעות של המודל מופיעה בין שתי מחלקות סמוכות של רמות מילוי גבוהות.

תוצאה זו מצביעה על כך שמודל XGBoost מצליח לבצע סיווג מדויק מאוד, אך קיימת רגישות מסוימת בהבחנה בין מחלקות מילוי קרובות.



איור 6: מטריצת בלבול עבור שילוב המודלים XGBoost ו-Random Forest

הגרף מציג את מטריצת הבלבול של המודל המשולב, המבוסס על שילוב הסתברויות החיזוי של XGBoost ו-Random Forest. ניתן לראות כי גם לאחר השילוב רוב הדגימות סווגו בצורה נכונה ונמצאות על האלכסון הראשי.

המחלקות 20%, 40%, 80% ו-100% סווגו באופן מלא, בעוד שבמחלקת 60% קיימת טעות אחת בלבד, שבה דגימה אחת סווגה כמחלקת 80%. תוצאה זו מעידה כי השילוב בין המודלים שומר על רמת ביצועים גבוהה מאוד ומספק מערכת יציבה לסיווג רמות מילוי המים.

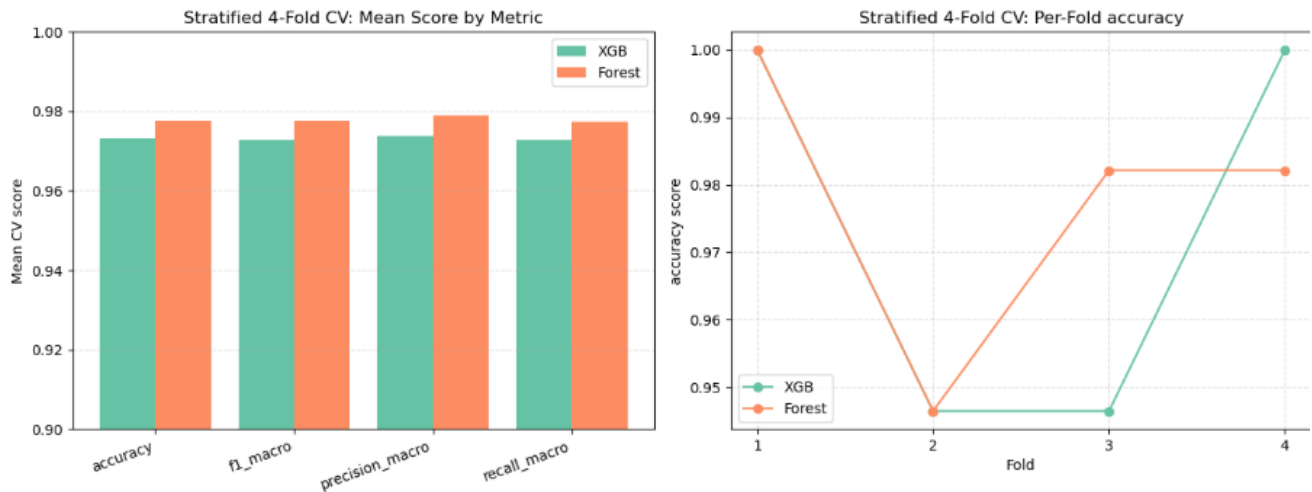
Combined				
	precision	recall	f1-score	support
20	1.00	1.00	1.00	12
40	1.00	1.00	1.00	11
60	1.00	0.91	0.95	11
80	0.92	1.00	0.96	12
100	1.00	1.00	1.00	11
accuracy			0.98	57
macro avg	0.98	0.98	0.98	57
weighted avg	0.98	0.98	0.98	57

איור 7: דוח מדדי ביצוע עבור המודל המשולב

האיור מציג את מדדי הביצוע של המודל המשולב עבור כל אחת ממחלקות המילוי. המדדים המוצגים הם Precision, Recall ו- F1-score לצד מספר הדגימות בכל מחלקה.

ניתן לראות כי עבור המחלקות 20%, 40% ו-100% התקבלו ערכים מושלמים של 1.00 בכל המדדים. עבור מחלקת 60% התקבל Recall של 0.91 ו- F1-score של 0.95, בשל דגימה אחת שלא סווגה כראוי. עבור מחלקת 80% התקבל Precision של 0.92 ו- F1-score של 0.96, מכיוון שדגימה ממחלקה אחרת סווגה אליה בטעות.

הדיוק הכולל של המודל המשולב עומד על כ-98%, וגם ממוצעי Macro Avg ו- Weighted Avg עומדים על כ-98%. תוצאה זו מעידה על ביצועים גבוהים, יציבים ומאוזנים בין המחלקות השונות.

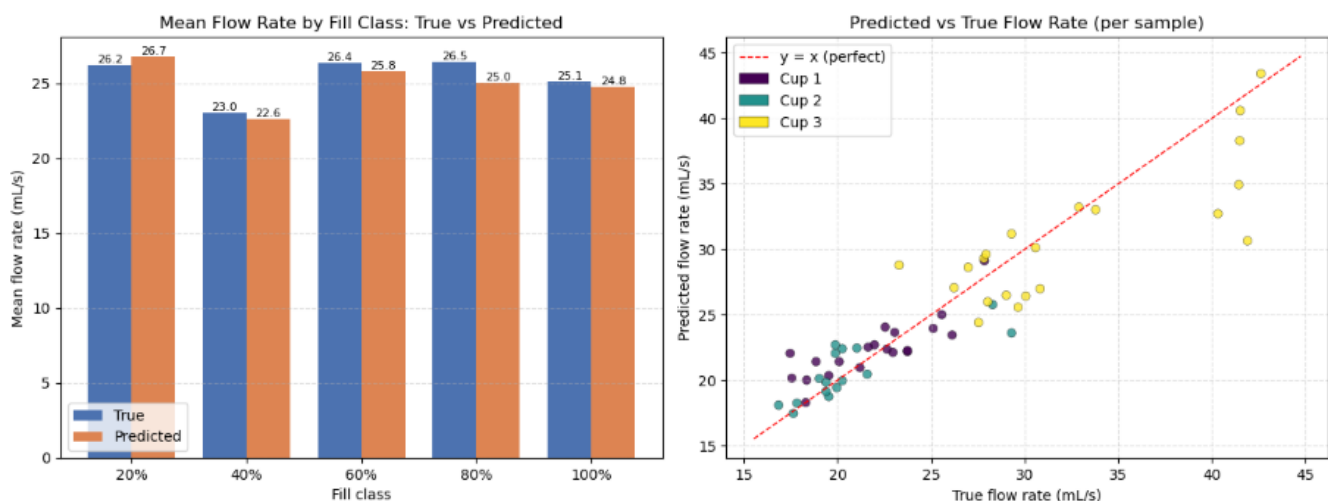


איור 8: השוואת ביצועי XGBoost ו-Random Forest באמצעות Stratified 4-Fold Cross-Validation

הגרף מציג השוואה בין ביצועי המודלים XGBoost ו-Random Forest באמצעות Stratified 4-Fold Cross-Validation. בגרף השמאלי מוצגים ממוצעי המדדים Accuracy, F1-score, Precision ו-Recall עבור שני המודלים.

ניתן לראות כי שני המודלים מציגים ביצועים גבוהים מאוד, מעל 97% במרבית המדדים. עם זאת, מודל Random Forest מציג יתרון קטן ועקבי על פני XGBoost בכל המדדים שנבדקו.

בגרף הימני מוצג דיוק המודלים בכל אחד מארבעת ה-folds. ניתן לראות כי שני המודלים מגיעים לביצועים גבוהים, אך קיימת ירידה מסוימת באחד מן ה-folds למרות זאת, Random Forest שומר על יציבות מעט גבוהה יותר, ולכן ניתן להסיק כי הוא המודל המתאים והיציב יותר עבור מאגר הנתונים בפרויקט.



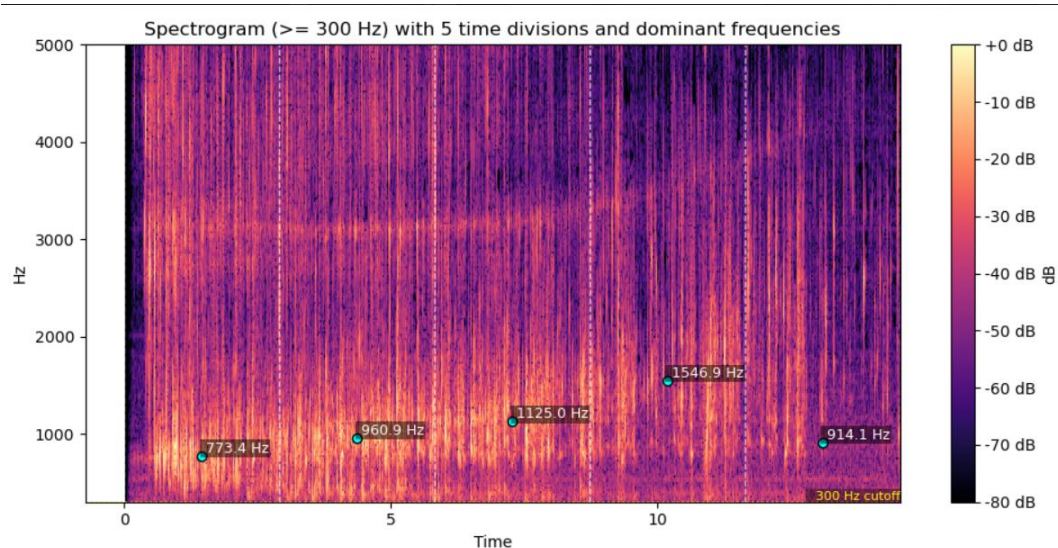
איור 9: השוואה בין קצב זרימת מים אמיתי לבין קצב זרימה חזוי

הגרף מציג השוואה בין קצב זרימת המים האמיתי לבין קצב הזרימה שנחזה על ידי המודל. בגרף השמאלי מוצג ממוצע קצב הזרימה עבור כל אחת ממחלקות המילוי, כאשר לכל מחלקה מוצגים הערך האמיתי והערך החזוי.

ניתן לראות כי קיימת התאמה גבוהה בין הערכים האמיתיים לערכים החזויים. במחלקות 20%, 40%, 60% ו-100% הפרשים בין הערכים קטנים יחסית. במחלקת 80% קיימת סטייה מעט גדולה יותר, אך עדיין נשמרת התאמה כללית טובה.

בגרף הימני מוצגת השוואה ברמת הדגימה הבודדת בין קצב הזרימה האמיתי לבין קצב הזרימה החזוי. הקו האדום מייצג חיזוי מושלם, שבו הערך החזוי שווה לערך האמיתי. ניתן לראות כי רוב הנקודות נמצאות בקרבת הקו, דבר המעיד על יכולת טובה של המודל לחזות את קצב הזרימה. עם זאת, עבור חלק מהדגימות, בעיקר בכוס 3 ובקצבי זרימה גבוהים יותר, קיימות סטיות גדולות יותר.

תוצאה זו מראה כי המודל מצליח ללמוד קשר בין המאפיינים האקוסטיים לבין קצב זרימת המים, אך קיימת רגישות מסוימת כאשר תנאי הזרימה משתנים בין סוגי הכוסות השונים.



איור 10 : ספקטרוגרמה וחילוץ תדרים דומיננטיים מתוך הקלטת האודיו
 הגרף מציג ספקטרוגרמה של אות האודיו לאחר סינון תדרים נמוכים, כאשר מוצגים תדרים מעל 300 Hz בציר האופקי מוצג הזמן, בציר האנכי התדר, והצבעים מייצגים את עוצמת האות בדציבלים.

ניתן לראות כי בארבעת המקטעים הראשונים קיימת עלייה בתדר הדומיננטי, מ-773.4 Hz ועד 1546.9 Hz, דבר המעיד על שינוי בצליל עם עליית מפלס המים בכוס. במקטע האחרון קיימת ירידה ל-914.1 Hz, ככל הנראה בשל רעש, שינוי בזרימה או תנאי ההקלטה. לכן, התדר הדומיננטי יכול לשמש כמאפיין חשוב, אך עדיף לשלבו עם מאפיינים נוספים.

סיכום ומסקנות כלליות

בפרויקט זה פותחה מערכת למידת מכונה לזיהוי רמת מילוי מים בכוס על בסיס ניתוח אותות אקוסטיים. במהלך העבודה נאספו 225 הקלטות משלושה סוגי כוסות שונים, כאשר כל הקלטה שויכה לאחד מחמישה טווחי מילוי. לאחר איסוף הנתונים בוצע עיבוד מקדים, חילוך מאפיינים אקוסטיים, בחירת מאפיינים, אימון מודלים והערכת ביצועים.

מהתוצאות עולה כי מאפיינים ספקטרליים, ובפרט Spectral Flatness ו- MFCC הם בעלי תרומה משמעותית להבחנה בין רמות מילוי שונות. בנוסף, נמצא כי מודל Random Forest השיג ביצועים גבוהים ויציבים יותר בהשוואה ל- XGBoost ולכן הוא נחשב למודל המתאים יותר עבור הנתונים שנאספו בפרויקט.

מטריצת הבלבול ותוצאות ה- Cross-Validation מצביעות על יכולת סיווג גבוהה מאוד בין חמש מחלקות המילוי. כמו כן, ניתוח קצב הזרימה הראה התאמה גבוהה בין הערכים האמיתיים לערכים החזויים. תוצאות אלו מחזקות את ההנחה כי ניתן להשתמש באות אקוסטי לצורך הערכת מצב המילוי של כוס.

מקורות וספרות עזר

1. חוברת הקורס " מבוא ללמידת מכונה"
2. Python Software Foundation — Python Documentation.
3. NumPy Documentation.
4. pandas Documentation.
5. librosa Documentation — Audio Feature Extraction.
6. Scikit-learn Documentation — Feature Selection.
7. Scikit-learn Documentation — Cross-Validation and Model Evaluation.
8. Scikit-learn Documentation — RandomForestClassifier.
9. XGBoost Documentation.
10. Matplotlib Documentation.
11. Seaborn Documentation.